Transfect : A cross project label prediction tool using transfer learning.

Software Project Lab 3 Technical Report

Submitted by
Khondaker Tasnia Hoque
BSSE-1205
Institute of Information Technology
University of Dhaka

Supervised by
Dr. Mohammad Shoyaib
Professor
Institute of Information Technology
University of Dhaka

Submission Date: 1 December, 2024



Letter of Transmittal

01 December, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - **Transfect :** A cross project label prediction tool using transfer learning.

Sir,
I am submitting the technical report for our Software Project Lab 3, "**Transfect :** A cross project label prediction tool using transfer learning."
This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,

Khondaker Tasnia Hoque
BSSE-1205
Institute of Information Technology
University of Dhaka

—----------

Supervisor's Signature

Acknowledgement

I am grateful to the Institute of Information Technology for giving me the opportunity to work on the project "Transfect: A Cross-Project Defect Prediction Tool." I would like to express my sincere gratitude to my supervisor, **Dr. Mohammad Shoaib**, Professor, Institute of Information Technology, University of Dhaka, for his constant guidance and support throughout the project. His valuable insights and expertise were crucial in shaping the development and outcomes of this work. Lastly, I would like to thank my classmates for their encouragement and constructive feedback, which greatly contributed to the progress of this project.

Abstract

This document outlines the requirements, specifications, architectural design, and testing of "Transfect: A Cross-Project Label Prediction Tool." The "Transfect" project introduces an advanced approach to cross-domain label prediction using transfer learning. By leveraging domain adaptation algorithms such as TCA, JDA, BDA, and CORAL, this tool facilitates efficient label prediction across projects with minimal labeled data in the target domain. Key components include data preprocessing, domain adaptation, and model evaluation using metrics like accuracy, precision, and F1-score. Additionally, it features an intuitive user interface, automated workflows, and comparative benchmarking to ensure robust and efficient performance.

List Of Figures **7**
List Of Tables **7**
1. Introduction **8**
1.1 Motivation 8
1.2 Scope 8
1.3 Project Timeline 9
1.4 Purpose of Document 9
2. Project Description **10**
2.1. Quality Function Deployment 10
2.1.1. Normal Requirements 10
2.1.2. Expected Requirements 11
2.1.3. Exciting Requirements 11
2.2 Usage Scenario 11
1. Installation 11
2. Data Upload 11
3. Input Method 11
4. Data Handling and Preprocessing 12
5. Model Training and Transfer Learning 12
6. Prediction and Result Display 12
3. Scenario Based Modeling **12**
3.1 Use Case Diagram 12
3.1.1 Transfect 13
3.1.2 Modules of Transfect 14
3.2.2.1. Input Data 14
3.2.2.2. Method Selection (Technical Perspective) 15
3.2.2.3. Data Processor 15
3.2.2.4. Feature Alignment 15
3.2.2.5. Label Prediction 15
3.2.2.6. Result Generator 16
4. Data Based Modeling **16**
5. Class-Based Modeling **16**
5.1 Analysis Classes 16
5.2 Class cards 17
6. Architectural Design **26**
6.1 Architectural Context Diagram 26
6.2 Archetypes 26
6.3 Top Level Components 28
6.4 Mapping Requirements to Software Architecture 29
7. Preliminary Test Plan **30**
7.1 High Level Description of Testing Goals 30
7.2 Test Cases 30
Test Case ID: TC001 30
Test Case ID: TC002 30

Test Case ID: TC003..... 30
Test Case ID: TC004..... 31
Test Case ID: TC005..... 31
Test Case ID: TC006..... 31
Test Case ID: TC007..... 31
Test Case ID: TC008..... 31
Test Case ID: TC009..... 32
Conclusion.....32
References :.....33

List Of Figures

Figure-1 : Timeline for Transfect……. 7
Figure-2 : UseCase Level 0 (Transfect) …….. 11
Figure-3 : UseCase Level 1 (Modules of Transfect)..... 12
Figure-4 : CRC Diagram of Transfect…… 23
Figure-5 : Architectural Context Diagram…….. 24
Figure-6 : Archetypes (Pipe and Filter)..... 25
Figure-7 : Archetypes (Client Server)..... 26
Figure-8 : Top Level Components ……..….. 26
Figure-8 : Requirements Mapping…….……. 27

List Of Tables

Table-1 : CRC Card (InputProcessor).....16
Table-2 : CRC Card (DataProcessor).....16
Table-3 : CRC Card (TransferLearningMethod).....17
Table-4 : CRC Card (TCA).....17
Table-5 : CRC Card (TCA+).....18
Table-6 : CRC Card (JDA).....18
Table-7 : CRC Card (BDA).....19
Table-8 : CRC Card (CORAL).....19
Table-9 : CRC Card (DeepCoral).....20
Table-10 : CRC Card (IPLS_DPLS_GDPLS).....20
Table-11 : CRC Card (KNNClassifier).....21
Table-12 : CRC Card (EvaluationMetrics).....21
Table-13 : CRC Card (ResultGenerator).....22
Table-14 : CRC Card (ErrorHandler).....22
Table-15 : CRC Card (UIHandler).....23

1. Introduction

The Transfect Tool is an advanced machine learning solution designed to address the challenge of adapting predictive models to new domains with minimal or no labeled data. Its core functionality is to train models on a well-labeled dataset from one project (source project) and apply them to another project within a similar domain (target project) where labeled data is scarce or unavailable. Here, a similar domain refers to projects that share a common context, such as software, industrial products, applications, or systems, with comparable features or attributes despite differences in data or project specifics.

Traditional machine learning models excel when applied to the specific contexts they are trained on but often fail to generalize effectively to new projects or datasets with differing characteristics. The Transfect Tool leverages state-of-the-art transfer learning algorithms to align feature distributions between source and target projects, enabling efficient and accurate label prediction. By addressing these challenges, Transfect provides a robust solution for cross-project predictive modeling, reducing reliance on extensive labeled data and improving model adaptability.

1.1 Motivation

In today's fast-paced and diverse development landscape, ensuring accurate and reliable predictions is crucial across various domains. However, the creation of labeled datasets for every new project or product is an expensive and time-consuming endeavor. This challenge is particularly evident in scenarios where projects share a common context but exhibit differences in data distributions or characteristics.Transfect aims to address these challenges by leveraging transfer learning, a powerful paradigm that allows models trained on one dataset (source domain) to adapt effectively to a different dataset (target domain) with minimal or no labeled data. By aligning feature distributions and generalizing knowledge across projects, Transfect facilitates faster, more efficient, and cost-effective label prediction. This approach significantly reduces the reliance on extensive labeling efforts while enhancing adaptability and performance in diverse applications, making it a versatile solution for a wide range of predictive tasks across industries.

1.2 Scope

The scope of **Transfect** is given below:

- **Transfect** is a desktop tool.
- It will predict labels of target dataset by following transfer learning methods:
    - TCA : Transfer Component Analysis
    - TCA+
    - JDA : Joint Distribution Adaptation
    - BDA : Balanced Distribution Adaptation
    - Coral : Correlational Alignment
    - Deep Coral
    - PLS : Projection to Latent Structures
    - Deep PLS
    - Generalized Deep PLS

1.3 Project Timeline

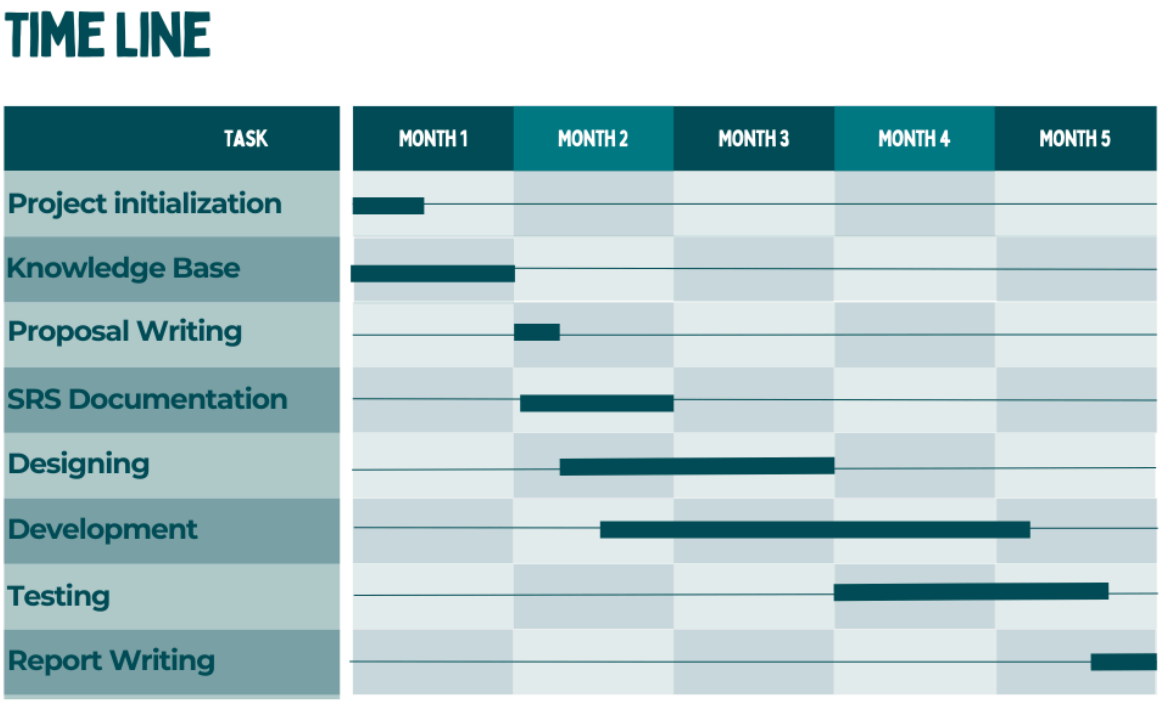


Figure-1: Timeline for Transfect

1.4 Purpose of Document

The purposes of this document are-

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

2. Project Description

Transferct will enable users to upload two files(source and target), choose any transfer learning method according to their preference and will show the predicted labels with some accuracy statistics. To achieve cross-domain prediction, this tool relies on preparing the data from both the source and target domains, ensuring that they are compatible for transfer learning. A variety of algorithms are applied to adapt the models trained on labeled data from the source domain to the target domain.

2.1. Quality Function Deployment

Quality Function Deployment (QFD) ensures that user needs and engineering characteristics are effectively translated into design and development specifications. The methodology allows the project team to prioritize features based on user requirements while optimizing functionality and usability. Below, we detail the Normal Requirements (NR), Expected Requirements (ER), and Exciting Requirements (ExcR) for each step in the Transfect Tool.

2.1.1. Normal Requirements

- NR1: Desktop Tool: The Transfect tool must be a desktop-based application, available for offline usage.
- NR2: Input Methods: Users can input source and target data through direct file upload. Support for csv, excel file formats
- NR3: Input Error Handle: System should handle the error regarding file inputs.
- NR4: Users can choose transfer learning methods :
    1. TCA
    2. TCA+
    3. JDA
    4. BDA
    5. PLS
    6. DPLS
    7. GDPLS
    8. CORAL
    9. DEEP CORAL.
- NR5: Visualization of performance metrics : Accuracy and F1-score.
- NR6 : Ability to handle large datasets efficiently.
- NR7 : Predict target project label.

2.1.2. Expected Requirements

- ER1 : Graphical User Interface (GUI) Controls: The GUI will provide drop down buttons for users to choose the method they wish to use for label prediction. Customizable algorithm selection based on user preference.
- **ER2 : Classifier Accuracy:** High accuracy and reliability in cross-project label prediction.
- ER3 : Input files from local file system
- ER4 : Compatibility with various domains and data distributions.

2.1.3. Exciting Requirements

- ExcR1 : The predicted label and actual label will be shown together so that the accuracy can be easily visualized.
- ExcR2 : Scalability to include deep learning models like Deep CORAL in future updates.
- ExcR3 : AUC and ROC curves, making it easier for users to track the effectiveness of their model in real-time.

2.2 Usage Scenario

The Transfect tool is designed to provide an intuitive solution for applying transfer learning to perform label prediction across different projects in similar domains. Below is the updated usage scenario:

1. Installation

- **Frontend**: Users can clone or download the **Transfect** repository and install dependencies using npm install. To start the React frontend, users simply run npm start.
- **Backend**: The Flask backend can be set up by installing the required dependencies listed in the requirements.txt file using pip install -r requirements.txt.
- **Start the Application**: Both the Flask server and React frontend are started simultaneously, providing the user with a full-stack web interface for label prediction across projects.

2. Data Upload

- **Source Data**: Users upload a labeled dataset (Excel/CSV format) from the source project. This dataset should contain both features and labels.
- **Target Data**: Users upload an unlabeled dataset from the target project (Excel/CSV format) that shares a similar domain with the source project but lacks labels.
- **Supported Formats**: Excel (.xlsx, .xls), CSV (.csv).

3. Input Method

- **Source and Target Data Setup**: Through the React interface, users select the columns in the source and target datasets that represent features and labels (for source data) or features (for target data).
- **User-friendly UI**: The app provides dropdowns and input fields for users to easily map the appropriate columns from the datasets.

4. Data Handling and Preprocessing

- **Data Preprocessing**: Transfect automatically preprocesses the datasets by handling missing data, scaling features, and applying necessary transformations.
- **Feature Matching**: The tool ensures that features from the source and target datasets are compatible, performing necessary alignment and adjustments.
- **Normalization**: The tool applies normalization and scaling techniques to bring the feature distributions of the source and target datasets closer together.

5. Model Training and Transfer Learning

- **Model Selection**: Users can choose from various transfer learning approaches or let the tool automatically select the most appropriate method based on the data characteristics.
- **Training Process**: The tool uses the labeled source dataset to train a predictive model. Transfer learning algorithms are then applied to adapt the model to the unlabeled target dataset, enabling label prediction.
- **Cross-Project Adaptation**: The tool adapts the model trained on the source data to perform label prediction on the target data, accounting for differences in data distributions across the projects.

6. Prediction and Result Display

- **Label Prediction**: After the model is trained, it predicts the labels for the target dataset, utilizing the knowledge transferred from the source project.
- **Performance Metrics**: The tool computes key metrics such as accuracy, precision, recall, and F1-score to evaluate the quality of the predictions.
- **Real-Time Display**: The prediction results and performance metrics are displayed directly in the React interface, allowing users to assess the model's accuracy on the target dataset.
- **Visualization**: Users can view performance charts, such as a confusion matrix or ROC curve, directly on the UI.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

3.1.1 Transfect

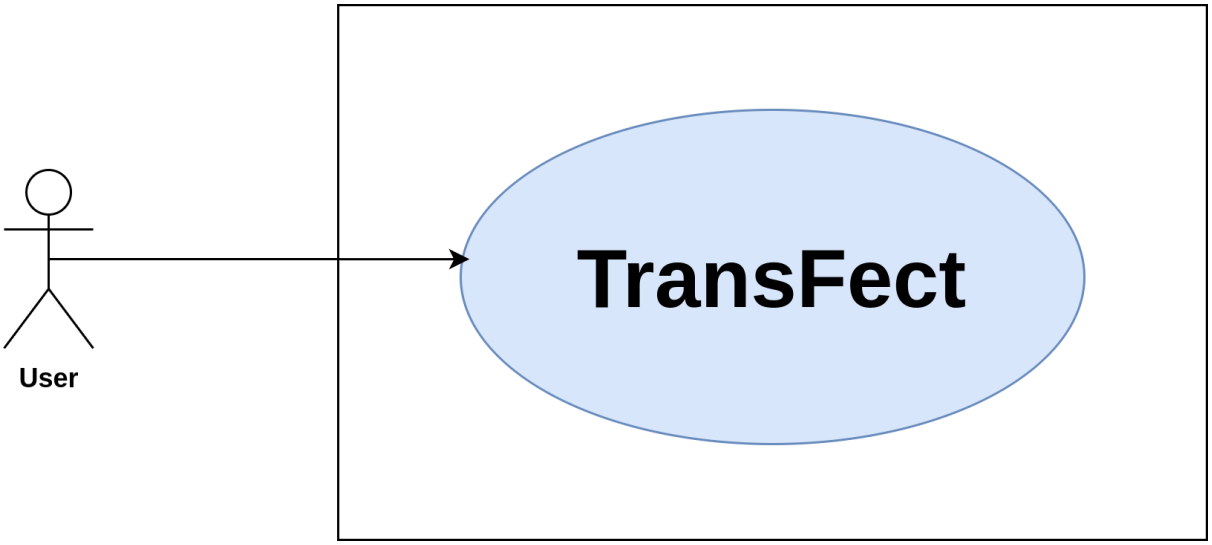


Figure-2: UseCase Level 0 (Transfect)

Primary actors: User
Secondary actors: No secondary actor
Goal in Context: The diagram shown above represents the whole Transfect tool.

3.1.2 Modules of Transfect

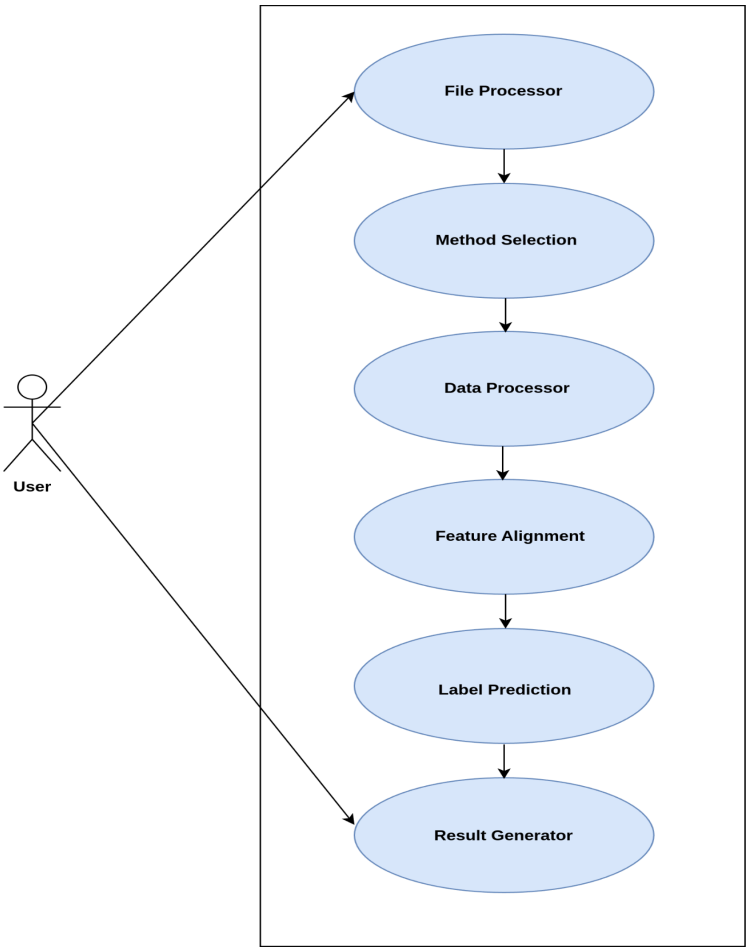


Figure-3: UseCase Level 1 (Modules of Transfect)

Primary Actor: User
Secondary Actor: No secondary actor
Goal in Context: The diagram shows the modules of Transfect. Transfect consists of 6 modules. The modules can be elaborated as follows

3.2.2.1. Input Data

The **Input Data** module is the entry point for user interaction. It allows users to upload datasets for both the **source project** (with labeled data) and the **target project** (with limited or no labeled data). Users can upload data in common formats such as **Excel**, **CSV**, or other supported formats. This module ensures the proper intake of data to facilitate subsequent processing.

3.2.2.2. Method Selection (Technical Perspective)

The **Method Selection** module in the backend is responsible for processing the user's choice of transfer learning technique, which is made via the front-end interface.

1. **User Input on Front-End**
   The user selects a transfer learning method (e.g., **TCA**, **CORAL**, **JDA**, etc.) through a dropdown menu or radio buttons in the React front-end application. Upon making the selection, an HTTP request (typically a **POST** or **GET** request) is sent to the backend Flask server. This request contains the chosen method as part of the payload (typically in the request body or query parameters).
2. **Backend Handling of Request**
   On the backend, the Flask server receives the request and extracts the selected method from the data. The server validates the selection to ensure it matches one of the supported methods (e.g., **TCA**, **CORAL**, **JDA**). If the selection is valid, the server proceeds to apply the corresponding transfer learning algorithm to the source and target datasets.

3.2.2.3. Data Processor

The **Data Processor** module performs essential pre-processing steps on the uploaded datasets. It includes:

- **Data and Label Extraction**: Extracts features and labels from the source and target datasets.
- **Normalization**: Standardizes or normalizes the data to ensure that all features have the same scale.
- **Cleaning**: Removes any noise or irrelevant data.
- **Transposing**: Adjusts the shape of data (e.g., converting columns to rows) to fit the model requirements.

3.2.2.4. Feature Alignment

The **Feature Alignment** module applies transfer learning techniques to adapt the source dataset to the target dataset. This step involves aligning feature distributions to minimize domain shifts between source and target projects. Transfer learning models are applied here to ensure that the source data's patterns are transferable to the target dataset. The selected transfer learning method ensures that feature distributions between the source and target datasets are aligned, making it possible to transfer knowledge effectively. This improves the prediction accuracy of the target project's labels. Once the data has been adapted by the chosen transfer learning algorithm, it is then processed further by the next module (Label Prediction).

3.2.2.5. Label Prediction

The **Label Prediction** module uses **K-Nearest Neighbors (KNN)** to predict the labels for the target project after the domain adaptation.

3.2.2.6. Result Generator

The **Result Generator** module compiles and presents the results of the prediction process. It includes:

- **Predicted Labels**: Displays the predicted labels for the target dataset.
- **Evaluation Metrics**: Provides performance metrics such as **F1-score**, **Accuracy**, and **AUC-ROC** to evaluate the effectiveness of the predictions. The module outputs these metrics to help the user assess the model's performance on the target dataset.

4. Data Based Modeling

The proposed tool will not be using a database or storage system. So this section will not contain any relevant analysis.

5. Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

After identifying nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes:

- InputProcessor
- DataProcessor
- TCA
- TCA+
- JDA
- BDA
- CORAL
- DEEP CORAL
- PLS_DPLS_GDPLS
- KNNClassifier
- EvaluationMetrics
- ResultGenerator
- UIHandler

Since all the **transfer learning models** (like TCA, CORAL, JDA, etc.) share similar responsibilities and are interchangeable, we can apply the **Strategy Pattern**. The **MethodSelector** class will choose the desired transfer learning approach and apply it. The final derived classes will look like this:

- TransferLearningMethod (Generic Class)
- TCA
- CORAL
- JDA
- BDA

In summary, the derived classes in the system are:

- InputProcessor
- DataProcessor
- TransferLearningMethod
- TCA
- TCA+
- JDA
- BDA
- CORAL
- DEEP CORAL
- PLS_DPLS_GDPLS
- KNNClassifier
- EvaluationMetrics
- ResultGenerator
- UIHandler
- ErrorHandler

5.2 Class cards

The class cards of SafeDep is given below:

InputProcessor	
Attributes	**Methods**
-sourceFIle -targetFile -methodType	+upload_sourceData() +upload_targetData() +Select_method() +handleSubmit()
Responsibility	**Collaborator**
● Handles file input from the frontend (e.g., source and target datasets). ● ● Handles method selection by user. ● Sends datasets and method type in the backend.	● ErrorHandler ● DataProcessor

Table-1: CRC Card (InputProcessor)

DataProcessor	
Attributes	**Methods**
-source_data_df -target_data_df -source_data -target_data -source_label -target_label -transposed_sourceData -transpose_targetData	+loadData() +processFIle() +normalization() +transpose()
Responsibility	**Collaborator**
● Extract data and labels ● Normalize data if needed ● Transpose datasets to match dimension	● ErrorHandler ● TransferLearningMethod

Table-2: CRC Card (DataProcessor)

TransferLearningMethod	
Attributes	**Methods**
-final_x_score -final_y_score -transformed_sourceData -transformed_targetData	+kernel() +fit() +pls() +dpls() +gdpls() +tca() +tca+() +jda() +bda() +coral() +deepcoral()
Responsibility	**Collaborator**
● Apply domain adaptation ● Feature extraction ● Transfer knowledge from source to target ● Overall cross domain feature alignment	● TCA ● TCA+ ● JDA ● BDA ● CORAL ● DeepCoral ● PLS_DPLS_GDPLS ● KNNclassifier

Table-3: CRC Card (TransferLearningMethod)

TCA	
Attributes	**Methods**
-kernel_type -dimension -gamma -Xs -Xt -MMD -CenterMetrics -eigenVector -eigenValue	+tca_fit()
Responsibility	**Collaborator**
● Implements **Transfer Component Analysis (TCA)** for feature alignment. ● Transforms source and target features into a common latent space.	● KNNClassifier

Table-4 : CRC Card (TCA)

TCA+	
Attributes	**Methods**
-similarity_vector[] -nomialValue_Cs[] -nomialValue_Ct[] -dist_mean -dist_median -dist_min -dist_max -dist_std -num_instances -mean_src -mean_tar -std_src -std_tar	+assign_nominal_values() +compute_characteristic_vector() +apply_normalization() +normalization_NONE() +normalization_N1() +normalization_N2() +normalization_N3() +normalization_N4()
Responsibility	**Collaborator**
● Extends TCA by incorporating additional normalization and feature scaling. ● Handles more advanced alignment scenarios for complex datasets.	● KNNClassifier

Table-5 : CRC Card (TCA+)

JDA	
Attributes	**Methods**
-list_acc[] -kernel_type -dimension -gamma -iteration -Xs -Xt -Y_tar_pseudo -MMD -CenterMetrics -eigenVector -eigenValue	+jda_fit()
Responsibility	**Collaborator**
● Implements **Joint Distribution Adaptation (JDA)** for aligning marginal and conditional distributions of source and target datasets.	● KNNClassifier

Table-6 : CRC Card (JDA)

BDA	
Attributes	**Methods**
-list_acc[] -kernel_type -iteration -mode -estimate_mu -dimension -gamma -Xs -Xt -MMD -CenterMetrics -eigenVector -eigenValue	+bda_fit()
Responsibility	**Collaborator**
• Implements **Balanced Distribution Adaptation (BDA)** to handle imbalanced data distributions between source and target domains.	• KNNClassifier

Table-7 : CRC Card (BDA)

CORAL	
Attributes	**Methods**
-Xs -Xt -cov_src -cov_tar -Xs_new	+coral_fit()
Responsibility	**Collaborator**
• Implements **CORrelation ALignment (CORAL)** for minimizing distribution differences between source and target datasets. • • Apply changes only on source based on covariance of target	• KNNClassifier

Table-8 : CRC Card (CORAL)

DeepCoral	
Attributes	**Methods**
-Xs -Xt -cov_src -cov_tar -Xs_new -learning_rate -weight_decay -momentum -batch_size -epochs	+loss_func() +coral() +fully_connectedCoral()
Responsibility	**Collaborator**
Extends CORAL to deep learning-based applications, leveraging neural networks for feature alignment.	• KNNClassifier

Table-9 : CRC Card (DEEP CORAL)

PLS_DPLS_GDPLS	
Attributes	**Methods**
-component -solver -max_iter -x_loadings -x_score -x_weight -y_loadings -y_score -y_weight -latent_vector_dimension -non_linear_mapping -nys_gamma -stack_previous_lv -mapping_func[] -layer	+pls_fit() +dpls_fit() +get_final_score()
Responsibility	**Collaborator**
• Implements Partial Least Squares (PLS), Discriminative PLS (DPLS), and Generalized DPLS (GDPLS) for feature alignment and dimensionality reduction.	• KNNClassifier

Table-10 : CRC Card (PLS_DPLS_GDPLS)

KNNClassifier	
Attributes	**Methods**
-y_pred	+KNeighborsClassifier() +predict()
Responsibility	**Collaborator**
● Performs label prediction using **K-Nearest Neighbors (KNN)** on aligned target features. ● Handles model training and testing on transformed datasets.	● EvaluationMetrics ● ResultGenerator

Table-11 : CRC Card (KNNClassifier)

EvaluationMetrics	
Attributes	**Methods**
-accuracy -auc_roc -f1	+accuracy_score() +auc_roc_score() +f1_score()
Responsibility	**Collaborator**
● Computes performance metrics such as accuracy, F1 score, precision, recall, and AUC-ROC. ● Provides a comprehensive evaluation of prediction results.	● ResultGenerator

Table-12 : CRC Card (EvaluationMetrics)

ResultGenerator	
Attributes	**Methods**
-targetData_with_label -targetData -targetLabel -predicted_label	+mergeData_Label
Responsibility	**Collaborator**
● Compiles prediction outputs and evaluation metrics into a presentable format. ● Sends results back to the frontend for display.	● UIHandler

Table-13 : CRC Card (ResultGenerator)

ErrorHandler	
Attributes	**Methods**
-error_msg	+file_validation() +method_validation +size_matching()
Responsibility	**Collaborator**
● Validates file formats (Excel, CSV). ● Validates transfer learning method type ● Validates source and target matrix size. ● Send error message	● UIHandler

Table-14 : CRC Card (ErrorHandler)

UIHandler	
Attributes	**Methods**
-inputData -resultData	+guiController() +eventHandler() +updateGUI()
Responsibility	**Collaborator**
• Manages the interaction between the frontend and backend. • Handles API calls from the frontend and sends data processing and prediction results back to the UI. • Ensures smooth user experience and error handling on the interface.	• InputProcessor • ResultGenerator

Table-15 : CRC Card (UIHandler)

The CRC diagram of the classes can is given below:

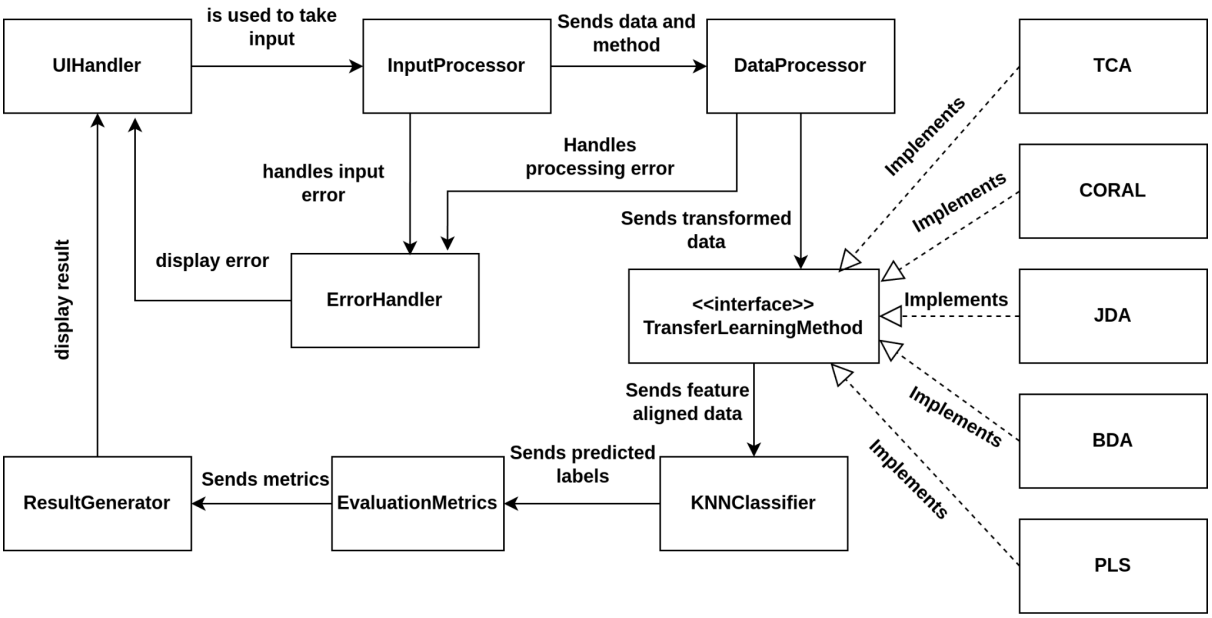


Figure-4: CRC Diagram of Transfect

6. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Diagram

It is possible to model how software communicates with entities outside of its boundaries using an architectural context diagram. The target system's cooperating systems are denoted by the following symbols:

Superordinate systems: Those systems that use the target system as part of some higher-level processing scheme. The proposed project has no superordinate systems.

Subordinate systems: Those systems that are used by the target system and provide data or processing that are necessary to complete the target system functionality. There are no subordinate systems in our project.

Peer-level systems: Those systems that interact on a peer-to-peer basis (i.e., information is either produced or consumed by the peers and the target system. There are no peer-level systems in our project.

Actors: Entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing. The user of our system is the Actor in this case. Each of these external entities communicate with the target system through an interface (the small shaded rectangles).

The architectural context diagram for the tool is given below :

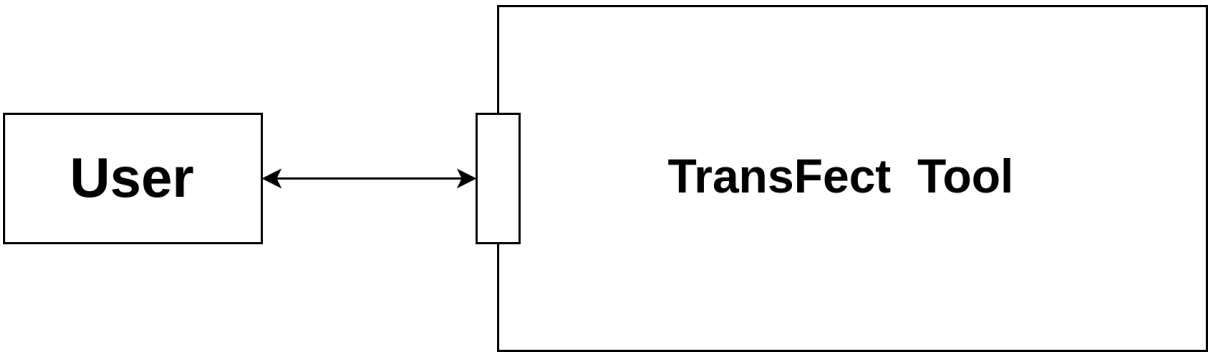


Figure-5: Architectural Context Diagram

6.2 Archetypes

The archetypes for the tool are defined below

An archetype refers to a class or template representing a fundamental concept crucial for shaping the architecture of the intended system. In essence, even highly intricate systems can be devised by employing a relatively limited set of these archetypes. These archetypes represent the enduring elements of the desired system structure, although they can take on different forms depending on how the system behaves. Together,these archetypes constitute the architecture of the intended system.

Suitable Archetype: Pipe and Filter Architecture, Client-Server Architecture

The Pipe and Filter architecture involves a series of filters (independent components) connected by pipes (data channels). Data flows sequentially through the filters, where each filter performs a specific transformation (e.g., cleaning, sorting, analyzing). This architecture is ideal for systems requiring step-by-step data processing and transformations.

Filters: Perform a specific function or transformation on the data.

Pipes : Transfer data between filters in a sequential flow.

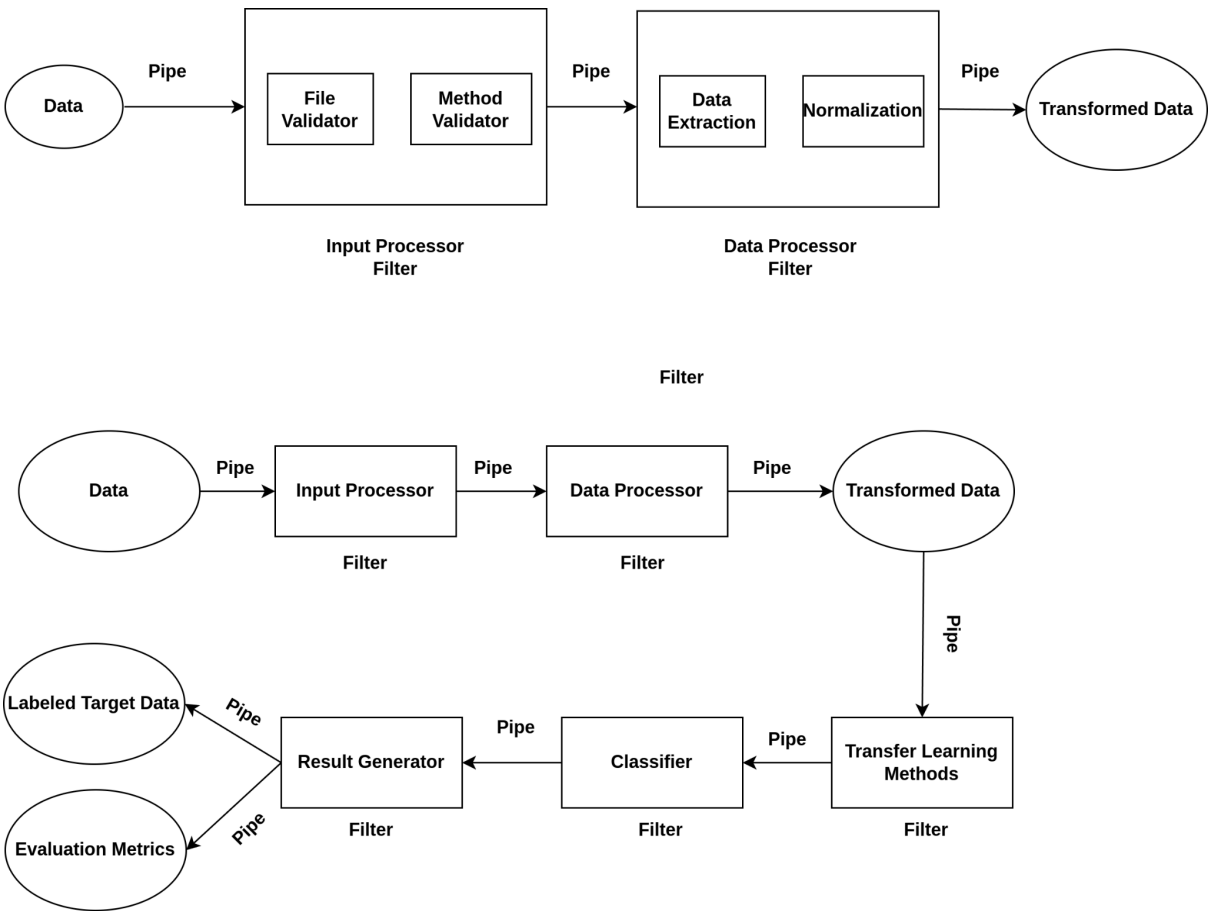


Figure-6: Archetypes (Pipe and Filter)

Client: The client is the front-end component, typically a user-facing application, that sends requests to the server. It handles user interactions, such as data input or visualization, and communicates with the server to process or retrieve data. Examples include web browsers or mobile apps.

Server: The server is the back-end component that receives requests from the client, processes them, and sends the appropriate response. It is responsible for tasks such as data storage, business logic, and application processing.

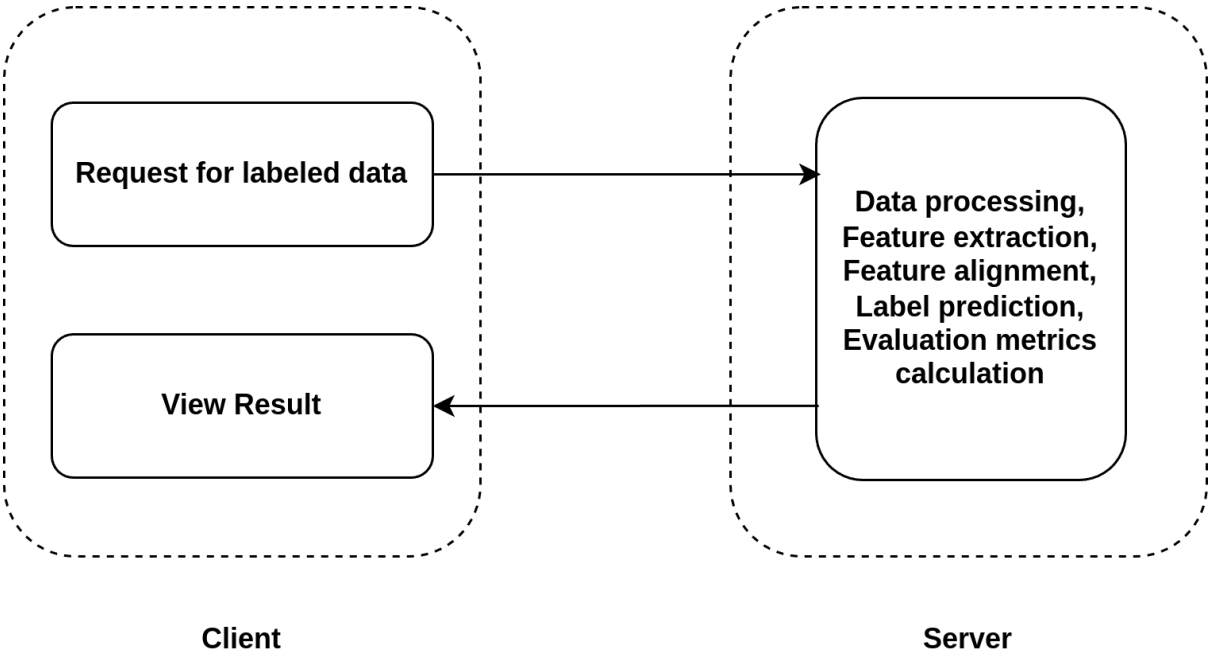


Figure-7: Archetypes (Client Server)

6.3 Top Level Components

Overall architectural structure with top level components is illustrated below

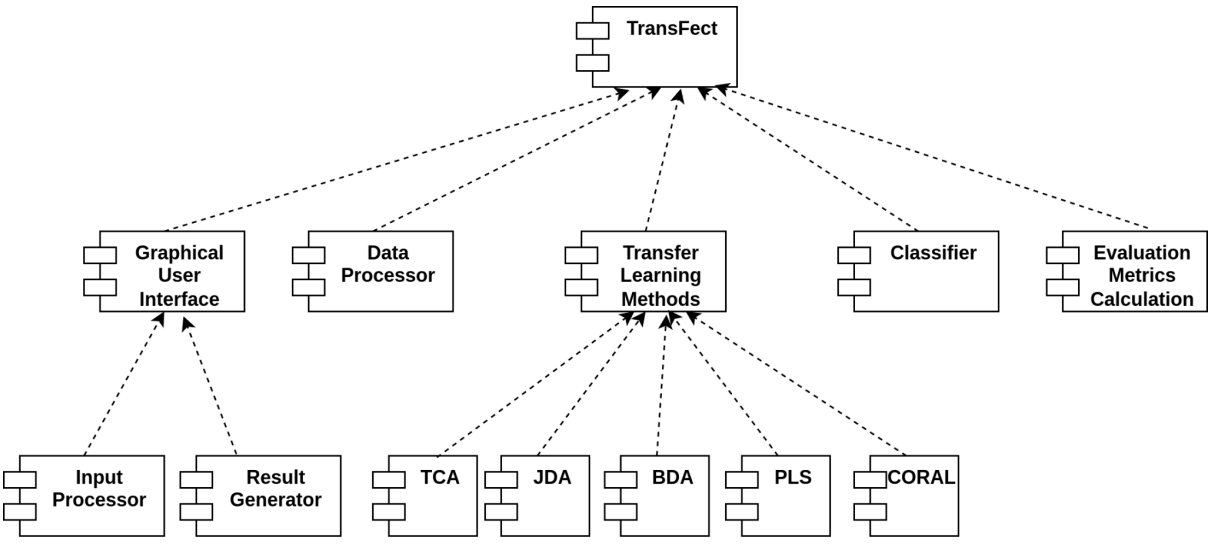
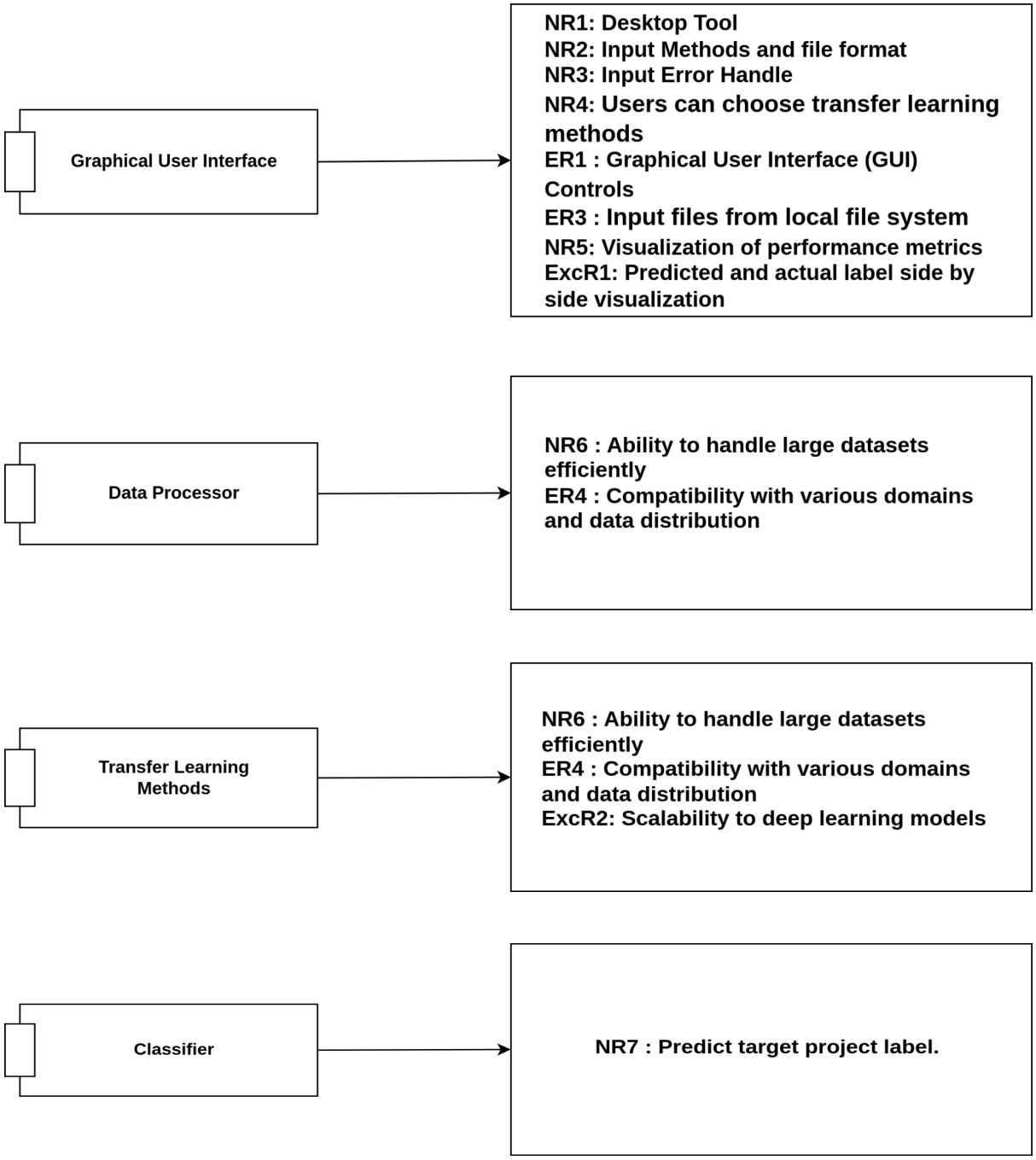


Figure-8: Top Level Components

6.4 Mapping Requirements to Software Architecture



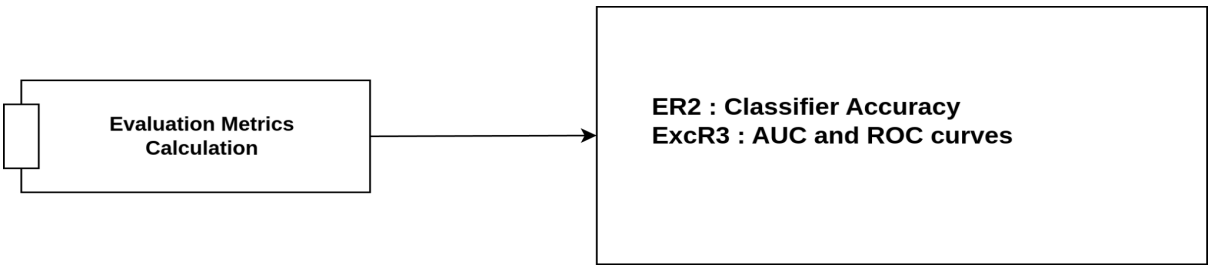


Figure-9: Requirements Mapping

7. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

7.1 High Level Description of Testing Goals

Testing goals for this tool is represented below from a high level-

- To demonstrate that the tool meets its requirements
- To ensure the features of npm packages are are extracted correctly
- To ensure that it produces correct detection most of the time
- To ensure the false positive and negative results are mitigated
- To ensure that it provides detailed analysis report
- To ensure that the tool receives feedback from the users properly
- To ensure that the tool runs smoothly

7.2 Test Cases

Test cases for the testing of this tool are documented below-

Test Case ID: TC001

Test Case Title: File Input Validation
Test Scenario: Verify that the system accepts valid input files in specified formats (CSV, Excel) and rejects invalid or unsupported files.
Expected Outcome(s):

- Valid files are accepted and displayed for further processing.
- An error message is shown for invalid, unsupported, or corrupted files.

Test Case ID: TC002

Test Case Title: File Input Error Handling
Test Scenario: Test the system's ability to handle empty or missing files during upload.
Expected Outcome(s):

- A clear error message is displayed when a file is missing or empty.
- The process does not proceed until a valid file is provided.

Test Case ID: TC003

Test Case Title: Transfer Learning Method Selection
Test Scenario: Verify that the user can select a transfer learning method from the dropdown menu.
Expected Outcome(s):

- The selected method is correctly passed to the backend for processing.
- An error is displayed if no method is selected and the user attempts to proceed.

Test Case ID: TC004

Test Case Title: Data Preprocessing Validation
Test Scenario: Verify that uploaded files undergo data preprocessing steps like cleaning, normalization, and transposing.
Expected Outcome(s):

- Data is cleaned and normalized accurately without altering the original structure.
- Errors are flagged if data preprocessing fails.

Test Case ID: TC005

Test Case Title: Method Execution for Feature Alignment
Test Scenario: Test the execution of selected transfer learning methods (e.g., TCA, CORAL) for feature alignment.
Expected Outcome(s):

- The selected method executes successfully and aligns source and target data.
- Errors or logs are shown if the execution fails.

Test Case ID: TC006

Test Case Title: Classification with KNN
Test Scenario: Verify that the KNNClassifier predicts target labels based on adapted data.
Expected Outcome(s):

- Predicted labels are generated for the target dataset.
- An error is displayed if the classification fails.

Test Case ID: TC007

Test Case Title: Evaluation Metrics Calculation
Test Scenario: Test the system's ability to calculate evaluation metrics such as accuracy, F1-score, AUC, and ROC.
Expected Outcome(s):

- Metrics are calculated and displayed accurately.
- Errors are flagged if calculations fail or inputs are insufficient.

Test Case ID: TC008

Test Case Title: Result Visualization
Test Scenario: Verify that the predicted labels, actual labels, and evaluation metrics are visualized correctly.
Expected Outcome(s):

- Results are displayed in a clear, user-friendly format.
- AUC and ROC curves are plotted accurately.

Test Case ID: TC009

Test Case Title: Large Dataset Handling
Test Scenario: Test the system's ability to process and analyze large datasets efficiently.
Expected Outcome(s):

- Large datasets are processed without performance degradation.
- Errors are flagged if the system exceeds resource limits.

Conclusion

In this report, I have provided a comprehensive overview of the **Transfect: A Cross-Project Label Prediction Tool Using Transfer Learning**. I have discussed the project's scope, assumptions, Quality Function Deployment (QFD), and usage scenario. Detailed scenario-based modeling, data modeling, and class-based modeling have been conducted to outline the tool's workflow and functionality. Additionally, the architectural design has been explained to give a clear understanding of the system's structure.

Furthermore, I have highlighted the high-level testing goals, along with the key features and items to be tested within the application. Figures and tables have been included to enhance understanding and facilitate a clearer grasp of the tool's technical aspects.

Though **Transfect** significantly reduces the dependency on labeled data across projects, further improvements can be made. Future developments could include incorporating more advanced transfer learning methods, expanding support to additional data formats, or optimizing the tool for broader domain applicability.

With this document, I have aimed to provide a well-rounded explanation of **Transfect's** design and functionalities. I hope this report serves to offer readers a clear and comprehensive understanding of how the tool operates, as well as its potential for cross-project label prediction.

References :

[1] B. Sun, J. Feng, K. Saenko. *Return of frustratingly easy domain adaptation.* Proceedings of the AAAI Conference on Artificial Intelligence, 2016.

[2] B. Sun, K. Saenko. *Deep CORAL: Correlation alignment for deep domain adaptation.* Computer Vision–ECCV 2016 Workshops: Amsterdam, The Netherlands, October 8–10, 2016.

[3] J. Nam, S. J. Pan, S. Kim. *Transfer Defect Learning.* 2013 35th International Conference on Software Engineering (ICSE), 2013.

[4] M. Long, J. Wang, G. Ding, J. Sun, P. S. Yu. *Transfer feature learning with joint distribution adaptation.* Proceedings of the IEEE International Conference on Computer Vision, 2013.

[5] X. Kong, Z. Ge. *Deep PLS: A lightweight deep learning model for interpretable and efficient data analytics.* IEEE Transactions on Neural Networks and Learning Systems, 2022.